An exact integer pre-filter for the NumberFields@home discriminant test

The Passiflora polDisc pre-filter: what it computes, why it is correct, and how that was checked

27 October 2026

Abstract

NumberFields@home tests very many monic degree-10 integer polynomials with an exact discriminant test that dominates the running time of the CPU application. Passiflora puts a pre-filter in front of it: a polynomial is dropped when, for one of a few primes q, the Legendre symbol of its discriminant modulo q is -1, which proves that it cannot pass. The pre-filter must never drop a polynomial that passes, or the project loses results. The first version computed the symbols in floating point and was exact only under round-to-nearest without x87 code; it lost valid polynomials when either failed. The present version uses only integer arithmetic (signed Montgomery reduction on the CPU, unsigned Montgomery reduction on the GPU). This paper states the algorithms, the theorems that make the decisions exact (the program text computes exactly the decisions of the exact algorithms, independently of the floating-point environment), the contract between the filter and the application, and the evidence: a test of the compiled filter on 25.5 billion polynomials, complete sets at tiny primes, an exact oracle with deliberate faults, a differential test of the exact test of the original GetDecics application, the 31 test work units in every build, and a shim of the GPU kernel. It also proves what that exact test computes, under one condition that the enumeration's search region is meant to guarantee and that the real data support: its first `int64` steps do not overflow. The proofs are in the full document [1]; here every result is stated with its status and with what it still assumes.

Contents

1 The problem **1**

2 The two chains and their theorems **2**

3 What the exact test of GetDecics computes **3**

4 Exact integer arithmetic **5**

5 The GPU filter **6**

6 The contract with the application **7**

7 The evidence **8**

8 What it all rests on **11**

1 The problem

NumberFields@home searches for number fields of degree 10 by enumerating monic polynomials $f = x^{10} + a_9x^9 + \cdots + a_0$ with integer coefficients.

GetDecics and Passiflora. The program that volunteers run for this is *GetDecics*, the project's original application [2]. Passiflora is not a new program: it is a patch on top of the original GetDecics application (source at commit `638ca4e`), plus new source files for the pre-filter and the GPU code. The enumeration and the exact discriminant test are GetDecics' own code; Passiflora decides which polynomials reach that test, and its output must be exactly that of the original application. In the full document [1] and in the repository, the original GetDecics application is called "upstream".

The final test of the enumeration is the *discriminant test* (`polDiscTest_cpu` of GetDecics, built on GMP):

Definition 1 (Passing)**.** Let S be a finite set of primes (read from the work unit; $S = \{2, 5\}$ in every test work unit). A monic $f \in \mathbb{Z}[x]$ *passes* if $D = \operatorname{disc}(f) \neq 0$ and $|D| = \big(\prod_{p \in S} p^{\alpha_p}\big) m^2$ for some integers $\alpha_p \geq 0$, $m \geq 1$.

Almost no polynomial passes, and the test is expensive (the repository reports that the stock CPU application spends 98% of its time in it [reported, not re-checked]). The pre-filter rests on one observation.

Proposition 2 (The rejection rule)**.** *[proved] Let f pass, and let q be a prime with $q \equiv 1 \pmod 4$ at which every $p \in S$ is a nonzero square. Then $\left(\frac{\operatorname{disc} f}{q}\right) \in \{0, 1\}$.*

Indeed $\chi(D) = \chi(\pm 1) \prod_p \chi(p)^{\alpha_p} \chi(m)^2$, $\chi(-1) = 1$ because $q \equiv 1 \pmod 4$, and $\chi(p) = 1$ by the choice of q. So a symbol -1 at one such prime is a proof that f fails. The CPU filter uses 30 such primes: 24 *small* ones below 2^{13} ($8161, 8089, \ldots, 6481$ for $S = \{2, 5\}$) and six *large* ones below 2^{25} ($33554249, \ldots, 33553649$). Each rejects about half of the candidates; survivors of all 30 go on to the exact test, which decides them as before. The GPU kernel applies the same filter at the 24 smallest such primes in $(256, 8192)$ ($281, \ldots, 1489$ for $S = \{2, 5\}$, none of them the CPU's) before it hands the survivors to the CPU.

Why integer arithmetic. The first version of the filter (commit `42c3270`) stored residues in floating point and reduced with the rounding trick $x - q \cdot ((x\,q^{-1} + M) - M)$, which is exact only if every operation is rounded to nearest in the declared precision. The fault guide of [1, Section 14] reproduces, with the real application, what happens otherwise [checked numerically]: built for x87 arithmetic (as a 32-bit x86 build is), the application lost valid polynomials in four of the seven smallest test work units (6, 12, 6 and 5), among them the only real result of one unit; a library that switches the rounding mode to "up" before the filter is initialised makes it reject valid polynomials; rounding down or toward zero makes it reject nothing. A filter whose correctness depends on the floating-point environment of the volunteer's machine cannot be proved correct for the program that ships. The integer version removes the dependence.

2 The two chains and their theorems

Everything here is over the field $\mathbb{F}_q$, q an odd prime with $q \nmid 10$; χ is the quadratic character. For monic f of degree n, $\operatorname{disc} f = (-1)^{n(n-1)/2} \operatorname{Res}(f, f')$, and the discriminant of f modulo q is the residue of $\operatorname{disc} f$ [proved]. The filter computes the symbol of $\operatorname{Res}(f, f')$ by Euclid's algorithm on f and f', without inverses: instead of dividing by a leading coefficient it multiplies by it (pseudo-division), and accounts for the powers it introduces. Only the parity of their exponents matters for a Legendre symbol [1, Section 4].

`chain_block`, the fast chain. For a *normal* remainder sequence of (f, f') (degrees $n, n-1, \ldots, 1, 0$) every step is two rounds with the same multiplier $\lambda = \operatorname{lc}(B)$: $R = \lambda A - \tau x B$ and $P = \lambda R - t B$, which equals $\lambda^2 (A \bmod B)$ (Lemma L2). The squares λ^2 never change the symbol, so the final constant has the symbol of the discriminant. The chain runs 64 polynomials in the same instructions (SIMD lanes) and checks, at each step, that the next leading coefficient is nonzero.

Theorem A (`chain_block` and its check)**.** *[proved] Let f be monic of degree $n \geq 2$ over $\mathbb{F}_q$, $q \nmid n$. The check passes at every step if and only if the remainder sequence of (f, f') is normal, and the first failing check is at the first abnormal degree. If it passes, the final constant `base` satisfies $\chi(\texttt{base}) = \chi\big((-1)^{n(n-1)/2}\big)\chi(\operatorname{disc} f)$; for $q \equiv 1 \pmod 4$ this is $\chi(\operatorname{disc} f)$, and a rejection means $\chi(\operatorname{disc} f) = -1$.*

`chain_general`, the general chain. Polynomials are stored left-aligned, so that one elementary step $W = l_v U - l_u x^{u-v} V$ is the same formula in every lane whatever the degrees; leading zeros of W are shifted out, U and V are swapped when $\deg W < \deg V$, and an accumulator `mm` collects l_v exactly at the steps whose exponent $e = \deg W - u + v$ is odd (by $\operatorname{Res}(V, W) = l_v^e \operatorname{Res}(V, U)$). It follows any degree sequence.

Theorem B (`chain_general`)**.** *[proved] Under the same hypotheses the algorithm stops within $2n - 2$ iterations (18 for $n = 10$; the code allows 40). Either a remainder is 0, and then $\operatorname{disc} f \equiv 0$ and the polynomial is kept; or it ends with a nonzero constant, and then $\chi(\texttt{mm}) = \chi\big((-1)^{n(n-1)/2}\varepsilon\big)\chi(\operatorname{disc} f)$ with a sign ε from the swaps. For $q \equiv 1 \pmod 4$ this is $\chi(\operatorname{disc} f)$, and a rejection means $\chi(\operatorname{disc} f) = -1$.*

The condition $q \equiv 1 \pmod 4$ is not decoration: with $q \equiv 3 \pmod 4$ the signs of the swaps make `chain_general` wrong in about a quarter of the degree-4 and 70% of the degree-10 cases tested [checked numerically].

The filter. For each prime in turn, every surviving polynomial goes through `chain_block`; the lanes whose check fails are re-run through `chain_general` (pass 2). A family of polynomials that keeps failing the check is sent to `chain_general` directly (`prefgen`), re-probed every 64 calls. The routing changes the speed, never a decision:

Theorem C (soundness and exactness)**.** *[proved] In exact arithmetic over $\mathbb{F}_q$: for every input, block size and routing state, `pdf_filter` gives a polynomial the flag 0 only if it is monic and has $\left(\frac{\operatorname{disc} f}{q}\right) = -1$ at some filter prime; so it never rejects a polynomial that passes. Conversely it rejects a monic polynomial exactly when some filter prime has symbol -1, at the first such prime.*

The converse is what makes a test with an exact oracle possible (Section 7).

3 What the exact test of GetDecics computes

The filter is proved against the definition of passing, but in the application its rejections take the place of verdicts of the original GetDecics application's own exact test, `polDiscTest_cpu`. So it matters what that test computes. It follows Cohen's sub-resultant algorithm [3, Algorithm 3.3.7]: starting from f and f' it computes a chain of pseudo-remainders whose last element gives the resultant $\operatorname{Res}(f, f')$, and so the discriminant, up to known factors. The numbers in that chain grow to about a thousand bits, so the test uses GMP; but to save time it does the first two iterations, where the numbers are still small, in 64-bit machine integers (`int64`), and only then switches to GMP. At the end it removes every factor of the primes of S from its result `polDisc` and asks whether what remains is a perfect square.

Proposition 3 (What the exact test computes)**.** *[proved] Let f be monic of degree 10 with `int64` coefficients, and suppose (H): no `int64` value computed in the first two iterations of `polDiscTest_cpu` leaves the `int64` range. Then $|\operatorname{disc} f| = r^2\,|\texttt{polDisc}|$ for a nonzero rational r, and `polDiscTest_cpu` sets the flag to 1 exactly when f passes.*

The proof [1, Proposition 3.19] follows the code of GetDecics step by step against Cohen's algorithm (whose exact divisions are quoted [standard result, quoted]): the content of f' divides 10 and is found exactly, every pseudo-remainder is the unique one by uniqueness of division in $\mathbb{Q}[x]$, the last step's shortcut replaces the resultant by an even power of a rational, and the removal of S is complete.

Hypothesis (H). The two `int64` iterations multiply coefficients of f by coefficients of f', and then by coefficients of the first remainder; the code does not check that these products fit into 63 bits, and relies on the polynomials it is given. The polynomials of the enumeration are bounded by its search region, and the author of GetDecics has said that polynomials generated for the fields under test should not overflow these steps. The source shows the same design: it sends one branch of the second iteration, where "the result would exceed a long", to GMP. For a polynomial outside that region the overflow is undefined behaviour in C; in practice the value wraps around modulo 2^{64}, the GMP steps that follow work on a wrong number, and the test answers "does not pass". (H) states the condition explicitly, so that the proposition says exactly what it rests on.

How it was measured. The program `test_poldisc.cpp` links the compiled `polDiscTest_cpu` of GetDecics and compares its flag with the definition computed independently by PARI (the discriminant, the primes of S removed, the square test). It also replays the `int64` steps of GetDecics in 128-bit integers, which shows whether one of them overflows. It runs eight families of polynomials: the real candidates of the application and the results of the test work units, and, to test the proposition itself, families built for the purpose (random coefficients, polynomials built to pass, square factors), which the enumeration does not produce:

Table 1: The compiled `polDiscTest_cpu` of GetDecics against the definition (PARI), and the Passiflora filter, on 10,735,681 polynomials ($S = \{2, 5\}$). "ovf": an `int64` step of GetDecics overflows; "differ": the flag of GetDecics is not the definition's.

		pass			differ		filter	
family	polynomials	GetDecics	definition	ovf	no ovf	ovf	rejects	rejects passing
random, $\lvert a_i\rvert \le 3$	3,000,000	458	458	0	0	0	2,930,945	0
random, $\lvert a_i\rvert \le 100$	2,000,000	0	0	0	0	0	1,999,948	0
random, $\lvert a_i\rvert \le 10^6$	1,000,000	0	0	999,961	0	0	1,000,000	0
built to pass, small	2,000,000	1,999,939	1,999,939	0	0	0	0	0
built to pass, large	1,000,000	155,735	1,000,000	844,277	0	844,265	0	0
square factor	1,000,000	0	0	0	0	0	0	0
real candidates	733,855	0	0	0	0	0	733,855	0
the 83 results, transformed	1,826	1,826	1,826	0	0	0	0	0
all	10,735,681	2,157,958	3,002,223	1,844,238	0	844,265	6,664,748	0

What it shows [checked numerically]. Wherever (H) holds, GetDecics agrees with the definition: on all 8,891,443 polynomials without an overflow, as the proposition says. The constructed families also show what (H) excludes: with coefficients far outside the search region (polynomials built to pass with $|G_k| \le 10^6$, random coefficients of 3000 and more) the `int64` steps overflow and the answer is no longer the definition's. Coefficient size alone is not the measure: the real candidates have coefficients up to $2.1 \cdot 10^9$ and never overflow, because their coefficients grow with the degree as those of polynomials with roots of moderate size do, which is what the search region produces. What matters is the largest value the `int64` steps form.

The evidence on real data. On the 146,771 candidates captured from seven real work units the largest `int64` value is $2^{59.45}$ (none above 2^{60}), a factor of about 12 below the limit 2^{63}; on the 83 results of the test work units it is $2^{48.5}$; and on all 733,855 real candidates and shifts of Table 1 no step overflows. This agrees with the author's statement. It is a measurement on samples, not a proof from the search bounds, so (H) remains a named assumption of this paper, with three and a half bits of measured headroom.

An illustration of the mechanism. The following polynomial is constructed; the enumeration does not produce it. It only shows what (H) excludes. One of the 83 results in the truth files of the test work units (which come with GetDecics) is

$$f = x^{10} + 600x^7 + 1350x^6 - 800x^5 + 115000x^4 + 405000x^3 + 265625x^2 - 540000x + 185000.$$

Multiplying its roots by 3 gives $g(x) = 3^{10} f(x/3)$,

$$g = x^{10} + 16200x^7 + 109350x^6 - 194400x^5 + 83835000x^4 + 885735000x^3 + 1742765625x^2 - 10628820000x + 10924065000.$$

It defines the same number field, and $\operatorname{disc} g = 3^{90} \operatorname{disc} f$, so g passes as f does (PARI: `poldisc`, `issquare`, `nfisisom`). Its largest `int64` value would be about 2^{67}, and GetDecics rejects it; with the roots multiplied by 2 the largest value is about $2^{56.5}$ and GetDecics accepts it. The enumeration produced f; scaled copies such as g, with roots three times as large, are not among the polynomials it generates.

Passiflora and (H). Passiflora keeps the exact test as GetDecics has it (its patch changes only the test's conversions for a 32-bit `long`), so that its output is the stock program's. The filter rejects only polynomials that provably do not pass, by the definition, so it never needs (H) for its own soundness: (H) enters only where the filter's verdicts are compared with those of the exact test (contract (C1)). Outside (H) the filter could differ from the stock program only by rejecting a polynomial that the exact test would wrongly accept; that did not happen in the tests (last column of the table).

4 Exact integer arithmetic

The representation. Elements of $\mathbb{F}_q$ are integers in a small range around 0. Products are reduced by signed Montgomery reduction: for $R = 2^k$, $qq' \equiv 1 \pmod R$ and $m = s_k(xq')$ (the signed residue in $[-R/2, R/2)$), $u = (x - mq)/R$ is an integer, $u \equiv xR^{-1} \pmod q$ and $|u| \le |x|/R + q/2$ [proved]. The small primes use $R = 2^{16}$ in 16-bit lanes (`mont`, 16 lanes per AVX2 register, compiled to `vpmulhw`/`vpmullw`), the large ones $R = 2^{32}$ with 64-bit products (`sredc`). At each prime, the coefficients of a block of 64 polynomials are copied from the application's buffer, and each 64-bit coefficient is split into three 21-bit pieces and loaded by one reduction (`ldcoef`, `ldres`).

What the language must do. The proofs use three rules: (S1) an operation whose exact result fits its kind gives that result; (S2) a conversion to a narrower kind reduces modulo 2^k; (S3) `shifta` is an arithmetic shift and `iand` with $2^k - 1$ keeps the low bits. The Fortran standard leaves (S2) and the bit model to the processor; the GNU Fortran manual documents (S3) but says nothing about (S2), which GCC documents for C. Standard-conforming replacements exist but made `pdf_filter` 2 to 6 times slower in a micro-benchmark (with the six small primes below 2^{10} it then had; AVX2: 105–126 ns per polynomial as written, 572–697 with `modulo` and exact division, 225–262 with offsets) [checked numerically], because the compiler then no longer uses the 16-bit multiply-high instructions. So the code keeps them, and they are checked in three ways: the compiled reductions against exact models on all inputs that matter (GCC 15.2 and 16.2), the whole filter on complete sets at tiny primes (Section 7), and at every start of the program (below).

Theorem D (the integer program is exact)**.** *[proved] Assume (S1)–(S3), and let the primes be those that `pdf_init` chooses. Then for every input buffer, block size and routing state, at every stage and in every lane, the code's decisions (the check of `chain_block`, `keep`) are those of the exact algorithms over $\mathbb{F}_q$ on $f \bmod q$. So Theorem C holds for the program as written. The filter executes no floating-point operation; its result depends neither on the rounding mode nor on x87 or SSE code nor on `-ffast-math`.*

The proof [1, Section 8] has four parts.

1. *Reductions and loading.* `mont` returns a value in $(-q, q)$ for operands in $(-2q, 2q)$ when $q \le 2^{13}$, `sredc` for $|x| < 2^{31}q$, and the three-piece loading gives $c \bmod q$ in $(-q, q)$ for every `int64` coefficient c; no operation overflows.
2. *Ranges.* Every value of the four chains stays where those lemmas apply: small-prime values in $(-2q, 2q)$ (a value is 0 mod q exactly when it is 0 or $\pm q$), large-prime values in $(-q, q)$.
3. *Weights.* Each polynomial the code holds is its exact counterpart times a power of R^{-1}, the same for all its coefficients, because both terms of every difference carry the same power. So every zero test, degree and swap agrees with the exact run, and as $R = 2^{16} = (2^8)^2$ and $2^{32} = (2^{16})^2$ are squares, no Legendre symbol changes. Euler's criterion runs in the Montgomery domain from $R \bmod q$, and -1 is recognised as the residue of $-R$.
4. *The masked loop.* `chain_general` runs all 64 lanes in one loop, masking the lanes that have finished and using a cheaper "lockstep" branch when all active lanes have the same degrees. A lemma [1, Lemma 8.10] proves, by induction over the iterations, that every lane computes the one-lane algorithm: entries beyond a degree are zero, a masked lane is never changed, the lockstep branch runs only when every lane is active with the same degrees, and copying the first $\max \deg + 1$ entries copies whole polynomials.

The size limits, and the guard. The single-reduction arguments stop at $q \le 2^{13}$ (small) and $q \le 2^{30} + 1$ (large). Following the bounds of the values through the chains instead gives larger limits for the whole filter, proved with a complete finite computation [checked numerically]: small primes up to 8385 and large primes up to 1,097,010,265. This holds although `mont` itself has counterexamples from 8241 and `sredc` on a difference from $2^{30} + 23279$, because the chains never form those operands. The limits are not known to be sharp. A directed search on a model of the chains [checked numerically], at every prime $\equiv 1 \pmod 4$ from 8389 to 19961 and at 64 large primes, then deeper below its first finds, gives the first results outside $(-q, q)$ at 15901 (small) and 1,749,999,961 (large), lower the longer it searches; the first wrong verdicts are at 18457 and 2,032,588,133, and those it found at primes `pdf_init` could choose make the real filter fail as well. So the proved limits are at least 0.53 and 0.63 of the true thresholds for such

failures, and probably not sharp; a search cannot show where in between the thresholds lie. The shipped primes are inside them, below 2^{13} and 2^{25}. `pdf_init` enforces them: a usable prime outside $[11, 8385]$ (small) or $[11, 1{,}097{,}010{,}265]$ (large) is skipped with one line on standard error, so no future change of the search can leave the proved region silently.

The start-up check. At the end of `pdf_init` the function `arith_ok` runs the compiled arithmetic on known inputs: `mont` and `sredc` on 64 inputs per prime including the extremes, the inverse relations $qq' \equiv 1$, the split of a coefficient into pieces, and the loads of the filter themselves on a block of 64 polynomials with these coefficients, all compared with values computed by `modulo` and exact division only, which do not depend on (S2) or (S3). If it fails, `pdf_init` writes one line on standard error and chooses no prime, so that every flag is 1 and the exact test alone decides [proved]. It guards against a compiler or flag set without the behaviour of (S2), (S3) on the machine where the program runs. It does not show that the code inlined into the chains was compiled the same way. The loads it runs are the compiled routines that load the polynomials (one copy of each in both objects), so a fault in the split, such as the deliberate fault "top piece shifted logically", fails it.

Instruction sets. The application contains an AVX2 build and a baseline (SSE2) build of the filter and picks the AVX2 one when the CPU reports AVX2 and FMA. The AVX2 build was compiled with `-march=x86-64-v3`, which also lets the compiler use BMI1/2, LZCNT, MOVBE and F16C, none of which the run-time test checks; it is now compiled with `-march=x86-64 -mavx2 -mno-popcnt`. The generated code is the same as before (114 distinct instructions, the same set as the `x86-64-v3` object, none outside AVX and AVX2 besides the base set) [checked numerically]; the change only makes the guarantee hold on paper as well.

5 The GPU filter

The GPU kernel runs the enumeration's innermost loops and, for every polynomial it counts, the one-lane `chain_general` at the small primes of its table. A polynomial it rejects is never shown to the CPU, so the kernel's filter must be sound by itself; survivors go through the CPU filter and the exact test as before. The earlier kernel used the single-precision floating-point trick. The present one uses unsigned 32-bit Montgomery arithmetic with $R = 2^{16}$, residues kept in $[0, 2q)$ (zero modulo q exactly when 0 or q), which needs no multiply-high instruction and no comparison per reduction.

Lemma 4 (`redc`)**.** *[proved] Let $q < 8192$ be odd, $q^- \equiv -q^{-1} \pmod{2^{32}}$, $0 \le t < 8q^2$, and $m = tq^- \bmod 2^{16}$. Then $t + mq < 2^{30}$ is divisible by 2^{16}, and $r = (t + mq)/2^{16} \equiv t\,2^{-16} \pmod{q}$ with $0 \le r < 2q$.*

Theorem E (the GPU filter is exact)**.** *[proved] For a prime $q \equiv 1 \pmod 4$ with $11 \le q < 8192$, the table row of q, and any `int64` coefficients $c_1, \ldots, c_{10}$, the kernel's `survives` rejects $f = x^{10} + c_1 x^9 + \cdots + c_{10}$ at q exactly when $\left(\frac{\operatorname{disc} f}{q}\right) = -1$. The table's primes are usable for S, so a polynomial the kernel rejects does not pass.*

The proof [1, Section 8.10] bounds every value passed to `redc` below $8q^2$, follows the weights as on the CPU, and runs Euler's criterion in the Montgomery domain. It uses only the semantics of unsigned integers in OpenCL C and CUDA, which wrap modulo 2^{32} by definition. A first integer version, with $R = 2^{32}$, canonical residues and `mul_hi`, was also exact but made the kernel 47% slower; the version above takes 9–12% less device time than the floating-point kernel (Table 3).

Host and device code must match. The theorem needs the kernel to read the table the program builds. For OpenCL this holds by construction (the kernel source is part of the program). The CUDA kernel is a fat binary embedded at build time and the MUSA kernel a library loaded at run time; a left-over binary of the floating-point kernel would read the new table wrongly and could reject passing polynomials, and the GPU self-check (which compares polynomial counts taken before the filter) would not notice. So the table layout has a number (format 2), the kernel's entry point carries it in its name (`nf_enum_t2`), and the MUSA library reports it and checks that every table row has the integer layout. A CUDA program with an old fat binary stops at start-up with a message; a MUSA program refuses an old library and runs on the CPU; an old program with a new library stops [checked numerically]. After

any change of the kernel the CUDA fat binary and the MUSA library must be rebuilt with the author's toolchain; this one requires that.

The filter on the device, at run time. At start-up every back end runs a self-test kernel of the same source once: `redc`, `coef_mod`, the verdict at every table prime for 240 polynomials and `survives`, 7,920 values for $S = \{2, 5\}$, which the host computes without the kernel's code (exact arithmetic modulo q, the symbol from the resultant); on a difference, or with device code that has no self-test, the task runs on the CPU. The window at the start of a task, where the CPU loops run next to the GPU, compares the counts; with `NF_GPUCHECK=1` (the whole task) the CPU also applies the kernel's own filter, compiled for the CPU, and the GPU's survivors must be the same, in the same order, or the task stops. That costs about a microsecond per polynomial and the first 5,000,000 polynomials have few survivors (0 to 10 in the test units), so the self-test is what checks the filter's values [checked numerically]: on the GTX 1650 it passed in the 12 GPU units at no measurable cost, a full comparison of one unit (3,759 survivors) found no difference, and injected faults and a kernel without the self-test were caught; the MUSA path was run with a stand-in runtime.

6 The contract with the application

What the application needs is stated as three properties.

(C1) Soundness. A flag is 0 only if the `polDiscTest_cpu` of GetDecics would reject the polynomial.

(C2) Safety. No integer overflow, out-of-bounds access, read of an undefined value, other undefined behaviour, or illegal instruction.

(C3) Integration. The flags reach the exact test as its `polyMask` argument expects, and the 31 test work units give their `.truth` files.

The interface [proved]. After `pdf_init(numP, pSet)`, `pdf_filter(pol, n, flag)` with $n \leq 0$ reads and writes nothing; with $n \geq 1$ it reads only the n rows of 11 coefficients, writes every flag with 0 or 1, and writes 0 only for a polynomial whose leading coefficient is 1 and which does not pass for $S = \{\texttt{pSet}\}$. Before the first `pdf_init`, and after a failed start-up check, every flag is 1; and the module state left by earlier calls changes the routing, never a flag. With Proposition 3 this gives (C1) under (H).

The glue [proved]. The application calls the filter into its flag array and then `polDiscTest_cpu` with the same array as `polyMask`. `polDiscTest_cpu` reads the mask at index k before it writes the flag at index k, touches no other index in that iteration, and writes a flag on every path; so afterwards the flag is 0 where the filter wrote 0 and the exact test's own flag elsewhere, which under (C1) is that flag everywhere: the output of the stock program. Every polynomial of the enumeration is monic (the enumeration sets the leading coefficient to 1 once), and the filter and the exact test get the same S.

Safety [proved]. Overflow: the range proofs; the call counter is kept modulo 64. Bounds: every index is at most n or the lane count, and the chains' indices stay in 0..11 (the masked-loop lemma). Undefined values: padding lanes hold x^{10}, a flag that `chain_general` does not set is set before it is read. Illegal instructions: the AVX2 object runs only when the CPU reports AVX2 and FMA, and contains nothing beyond AVX2. Language-level undefined behaviour: none apart from the conversions and shifts of (S2), (S3), which the start-up check tests.

The interface. (A1) At most three primes in S [proved]: GetDecics keeps S in `int pSet[3]`, and so do the patch's copies, each filled without a bound check; the program refuses a work unit with more primes right after it reads S, before any of them is filled (tested with four and five primes; all 31 work units have $|S| = 2$). Named assumptions: (A2) One thread: the filter keeps module state, and the application calls it only from the thread that runs the enumeration. (A3) Hypothesis (H) of Proposition 3.

7 The evidence

The theorems are about the program text and the mathematics; the compiler, the device and the code of GetDecics are trusted only as far as the checks below show. All CPU runs on Linux (WSL2, GCC and gfortran 15.2) unless stated.

The compiled filter on complete sets at tiny primes [checked numerically]. The filter was built from the shipped source with its prime search moved to one tiny prime $q \in \{13, 17, 29, 37, 41\}$ (only four constants changed), once in the 16-bit and once in the 32-bit stage, and every polynomial of a set was sent through the AVX2 and the baseline build, each with the normal routing and with every lane forced through `chain_general`: eight runs per polynomial. Every flag and every rejection stage was compared with an exact oracle (Euclid with inverses). The sets: all 13^9 depressed monic polynomials modulo 13 (every monic polynomial mod 13 is a shift of one of them); at 17 all with $a_9 = a_8 = 0$, at $29, 37, 41$ all with $a_9 = \cdots = a_6 = 0$ (which send every chain into `chain_general` and its lockstep branch); and at each prime 200,000 random polynomials with all their shifts and coefficients lifted by large multiples of q. Altogether 25,518,310,785 polynomials and 204,146,486,280 verdicts: none wrong. Both branches of the masked loop ran billions of times.

The arithmetic alone [checked numerically]. The reductions, cut out of the source (not retyped) and compiled with the filter's flags and at `-O0`, against exact models: `mont` on every pair of operands for every odd prime below 2^{13} (333,937,622,331 pairs) and every odd q in $[8193, 8303]$; `sredc` on windows around its limits and random inputs; the loading on 101,072,896 coefficients for 6,169 moduli; the bound propagation of the size limits for every odd q up to 32,767 (small) and 1,100,000,001 (large). With GCC 15.2 (Linux) and 16.2 (MSYS2): no error.

An exact oracle, and deliberate faults [checked numerically]. A test that only feeds polynomials that must pass cannot see a filter that rejects too little [proved]. The oracle test computes the symbol at every filter prime independently and builds its inputs to reach every stage and path of the filter, with metamorphic checks and a test hook that reports the stage of every rejection. Seven deliberate faults of the integer arithmetic, each one changed line, against 1,000,000 polynomials (oracle test total failures, AVX2 build / the old "must pass" test):

fault	oracle test	"must pass" test
none	0	0
zero test $\pm q$ in `chain_block_s`	360	0
zero test $\pm q$ in `chain_general_s`	23,283	349
Euler: $-R$ recognised only as one representative	57,624	0
top piece of a coefficient shifted logically (caught by the start-up check: filter off)	643,008 missed	0
wrong loading constants (caught by the start-up check: filter off)	643,008 missed	0
parity of `chain_general`	236,849	161,538
last stage skipped	7,280	0

The oracle test sees all seven; the floating-point faults (x87, rounding up, down, toward zero) give 0 on the integer filter, as Theorem D says, while on the floating-point filter of `42c3270` the oracle test saw 66 false and 296 missed rejections under x87 and 37,329 false under rounding up. Primes moved across the size limits behave as the proofs predict: no wrong verdict inside them; beyond them wrong verdicts appear, at some primes only for directed inputs found by a search. With 24 small primes the generated polynomials reach the large stages only through the constructions that control every symbol; the large-stage code is the same, instruction for instruction (`objdump`), as in a build with six small primes below 2^{10}, which the oracle test ran with 788,246 of 1,000,000 polynomials reaching the large stages and no failure. The 146,771 candidates captured from seven real work units pass through the filter with the oracle's verdicts.

The exact test [checked numerically]. Table 1.

The GPU filter [checked numerically]. The kernel's `survives`, compiled for the CPU from the kernel source itself, against the exact oracle prime by prime: 6,000,000 polynomials at the 24 table primes and 3,000,000 at tiny primes and just below the limit 8192, 228,000,000 checks, no false and no missed rejection. On an NVIDIA GTX 1650 (OpenCL, and CUDA with the kernel compiled by NVRTC), the application gave the truth files of 8 work units through OpenCL and 4 through CUDA, before and after the format check was added and again with the present table. The table used to take its primes below 1024 only, a bound of the floating-point kernel: 14 for $S = \{2, 5\}$, enough, but of the 833 sets of one to three primes below 60, 592 leave fewer than the 8 that GPU mode needs (it then runs on the CPU), and below 8192 none does. Smallest first, because Euler's criterion takes one step per bit of $(q-1)/2$: with the 24 largest primes the kernel took 1.0–1.3% longer, with the 24 smallest 0.0–0.3%, within the scatter of the runs.

The work units [checked numerically]. The application, built on Linux from GetDecics at `638ca4e` with the patch and the repository's compile lines (BOINC replaced by stand-ins), gave the `.truth` files of all 31 test work units with the AVX2 build and with the baseline build. With `NF_VERIFY=1` (the exact test also on every polynomial, stopping at the first difference), five units in both builds ran 20,008,960 polynomials each without a difference.

Speed [checked numerically]. On the GPU, Table 3 gives one series of the kernel in four versions: the floating-point kernel and the integer kernel in three steps, each with one change more than the step before (the integer arithmetic with the earlier table; the wider table; the start-up self-test). The versions alternate, and every run gave its truth file. The integer kernel takes 9–12% less device time than the floating-point one. The wider table adds 0.3–0.9%, and the self-test, which runs once at start-up, leaves the device time unchanged. A whole run takes 3–10% less on the three longer units; on `sf5`, where start-up and the window check take most of the run, the differences are inside the spread of the runs. Two earlier series had given about 10% (1217 against 1091 ms on `sf5`, 3715 against 3324 ms on `sf4`) and later 2720 ms for the same integer kernel on `sf4`; the first ran with the CPU fully loaded. On this machine, a laptop, that matters: with 16 threads spinning (CPU at 100%), the device time on `sf4` rose from 2651 to 3123 ms and fell back to 2666 ms afterwards, on `sf7` from 6975 to 7015 ms. The GPU's core clock stayed at its maximum, 1785 MHz, with no throttle reason, but the GPU was idle for longer stretches during those runs and its memory clock dropped to its idle 375 MHz at times, a likely cause that was not established. So the GPU times of Table 3 were taken with the CPU otherwise idle.

On the CPU, Table 2 gives the application's own timing split (`NF_PROFILE`) over the first 60,006,400 polynomials of three work units, seven runs per filter, the filters alternating, one run at a time on an otherwise idle Linux machine (one-minute load 0.19 before the first run, 1.04 after the last). The applications differ only in the filter object: the floating-point filter of `42c3270`, compiled as it shipped (`-march=x86-64-v3`); an earlier integer version with Barrett reduction (32-bit lanes for the small primes, 64-bit products for the large ones; never committed, it gave the truth files of all 31 work units in both builds before it was timed); and the Montgomery filter in three steps, each with one change more than the step before: as committed in `8e19075`, with six small primes below 2^{10} and 24 large ones $(6 + 24)$ and scalar loads; the same with vectorised loads; and these loads with 24 small primes below 2^{13} and six large ones $(24 + 6)$, the filter this paper describes (`9d9837e` made both changes). On `sf5` and `sf3` almost every polynomial takes `chain_block`; on `sf4` every polynomial takes `chain_general` at every prime (the stage counts of `NF_PDFDEBUG`).

In `8e19075` the small-prime loads were not vectorised: gfortran's cost model rejected the loop that gathered three arrays of 21-bit pieces through the block's indices, and cycle counters in the application showed these scalar loads (114 cycles per polynomial on `sf5`, against 61 in the floating-point filter) to be the whole of its gap with AVX2. The vectorised loads copy each polynomial's ten coefficients into a block buffer and split and reduce them on contiguous lanes, which gfortran vectorises, and the split once per buffer is gone (54 cycles per polynomial in the floating-point filter, 44 before). The arithmetic and the compiled chains are unchanged. The 24 small primes then save most of the large stages, which cost about twice as much as in the floating-point filter: a polynomial reaches them only if no small prime rejects it, and with 24 almost none does (cycle counters: 0.4 instead of 30.7 cycles per polynomial in the large stages, 6.2 more in the small ones, the filter 8.6% faster). The split $24 + 6$ had the lowest medians of those with 30 primes in two earlier timings ($6 + 24$ and $12 + 18$ below 2^{13}). The $24 + 6$ filter is faster than the floating-point one in every row: by 14% and 4% with AVX2 on `sf5` and `sf3` (the 4% is less than the spread of the runs: the fastest and the slowest run of one filter differ by 5–31%), by 19–20% with

the baseline build, and by 48–58% on `sf4`; against the vectorised loads with six small primes it is 7–15% faster. Timings of different series differ by up to about 10%, so only filters of one series are compared. The Barrett version is the slowest throughout: AVX2 has no 64-bit vector multiplication, and a 32-bit lane holds half as many residues per register as the 16-bit lanes of the Montgomery stage.

Why $24 + 6$ [checked numerically]: a model of the cost, measured stage by stage with cycle counters in the application (five work units, two of them from the two searches that are running; ten splits; both builds; five runs each), explains the split. Each stage keeps half of the polynomials whatever the size of its prime (0.5001 at the small primes, 0.4993–0.5007 at the large ones), a call stops as soon as it has no polynomial left, a large stage costs 3.1–4.2 times a small one per block, and the exact test costs 6.7–8.0 μs per kept polynomial. The model predicts the filter time of every split to within 1%. An exchange argument shows that small primes belong first [proved]; of the splits of 30 stages $30 + 0$ is the cheapest and $24 + 6$ is within 0.14% of it, and 30 stages are within 0.10% of the best number. All three prime sets of the project's searches ($\{2, 5\}$ now, $\{2, 11\}$ and $\{5, 11\}$ before) have more than 100 usable primes below 2^{13}, and the first stage alone takes almost half of the filter's time.

Table 2: CPU filter plus exact test (the application's polDisc time) in seconds for the first 60,006,400 polynomials, median of seven runs; in brackets the change against the floating-point filter. Montgomery: the three steps of the text, each with one change more: $6 + 24$ primes (small + large) with scalar loads (`8e19075`); the same primes with vectorised loads; then $24 + 6$ primes with these loads (`9d9837e`, the filter described here). Work units `sf5_DS-14x11_Grp33043of705883`, `sf3_DS-16x270_Grp74of3932160`, `sf4_DS-13x271-25_Grp98of200000`.

				Montgomery		
				scalar loads	vectorised loads	
work unit	build	floating point	Barrett	$6 + 24$ primes	$6 + 24$ primes	$24 + 6$ primes
`sf5`	AVX2	5.84	12.03 (+106%)	6.65 (+14%)	5.49 (−6%)	5.03 (−14%)
	baseline	8.86	24.47 (+176%)	8.80 (−1%)	7.96 (−10%)	7.21 (−19%)
`sf3`	AVX2	5.44	11.87 (+118%)	6.52 (+20%)	5.65 (+4%)	5.23 (−4%)
	baseline	8.20	22.52 (+175%)	7.89 (−4%)	7.33 (−11%)	6.59 (−20%)
`sf4`	AVX2	13.37	16.09 (+20%)	8.55 (−36%)	7.64 (−43%)	6.99 (−48%)
	baseline	21.88	32.81 (+50%)	11.25 (−49%)	10.86 (−50%)	9.28 (−58%)

Table 3: GPU filter on the GTX 1650 of the test machine (a laptop on mains power, CPU otherwise idle, GPU core clock at its maximum 1785 MHz whenever busy), one series, medians; in brackets the change against the floating-point kernel. "device": the device time of the filter kernel's launches (OpenCL, a build instrumented for this measurement, fixed launch size, window check off), 7 runs, spread 0.5–2.5%. "OpenCL", "CUDA": the whole run with the program's defaults, 5 runs, spread up to 13% on `sf5`, up to 8% on the others. Integer kernel: the three steps of the text, each with one change more: the earlier table, for $S = \{2, 5\}$ the 14 usable primes in $(256, 1024)$ (`62550dc`); the 24 smallest usable primes below 8192 (`70ef543`); the start-up self-test (`741523c`, the program described here). Floating point: `77d7318`. Work units as in Table 2 and `sf7_DS-12x12_Grp881552of1000000`.

			integer, Montgomery $R = 2^{16}$		
			14 primes below 2^{10}	24 primes below 2^{13}	
work unit	time	floating point	no self-test	no self-test	self-test
`sf5`	device (ms)	1191	1044 (−12.3%)	1054 (−11.5%)	1054 (−11.5%)
	OpenCL (s)	2.68	2.70 (+1%)	2.67 (0%)	2.62 (−2%)
	CUDA (s)	2.51	2.49 (−1%)	2.49 (−1%)	2.46 (−2%)
`sf3`	device (ms)	15029	13592 (−9.6%)	13627 (−9.3%)	13629 (−9.3%)
	OpenCL (s)	16.76	15.41 (−8%)	15.37 (−8%)	15.32 (−9%)
	CUDA (s)	18.04	16.69 (−7%)	16.74 (−7%)	16.77 (−7%)
`sf4`	device (ms)	3027	2662 (−12.1%)	2674 (−11.7%)	2692 (−11.1%)
	OpenCL (s)	7.97	7.55 (−5%)	7.56 (−5%)	7.61 (−5%)
	CUDA (s)	7.86	7.58 (−4%)	7.45 (−5%)	7.64 (−3%)
`sf7`	device (ms)	7902	6989 (−11.6%)	7013 (−11.2%)	7011 (−11.3%)
	OpenCL (s)	9.59	8.63 (−10%)	8.78 (−8%)	8.80 (−8%)
	CUDA (s)	9.52	8.94 (−6%)	8.89 (−7%)	8.99 (−6%)

8 What it all rests on

The full document ends with a ledger of every claim, how it was checked and what it still assumes; `checks.py` checks its form and every source-line citation of the document. Its contract rows and its named assumptions:

claim	status	rests on
(C1) soundness	[proved]	Theorems A–D, Proposition 3, the interface; assumes (H), compiler correctness, (S2), (S3)
(C2) safety	[proved]	range proofs, size limits, the masked-loop lemma; assumes (S2), (S3) (start-up check), the CPU's report of AVX2/FMA
(C3) integration	[proved] [checked numerically]	the glue proposition; 31 units in both builds, 12 on the GPU; assumes (A2)
GPU filter	[proved]	Theorem E; assumes the device compiler and device correct (tested on the device at start-up), host and device code of one format (checked at start-up)
compiler correctness	[named assumption]	tiny-prime test, arithmetic checks with two GCC versions, oracle test, start-up check; docs/BUILDING.md names the checked compilers
(S2), (S3)	[named assumption]	as above; the start-up check switches the filter off if either fails on the running machine
(H)	[named assumption]	the enumeration's search region (the author's statement); no overflow on 733,855 real candidates and shifts, largest value $2^{59.45}$ against 2^{63}
(A2)	[named assumption]	no thread of the program calls the filter

Open items.

- Whether the size limits 8385 and 1,097,010,265 are sharp (no shipped prime is near them). The directed search narrows the first failure to $(8385, 15901]$ and $(1{,}097{,}010{,}265,\ 1{,}749{,}999{,}961]$ for primes $\equiv 1 \pmod 4$; closing the gap needs a sharper proof, not more search.
- (H) for the application's polynomials is measured on real candidates (about 12 times headroom), not proved from the search bounds.
- The GPU filter is checked on the CPU and on one NVIDIA card; AMD, Intel and MUSA devices, and the CUDA kernel built by `nvcc`, were not tested; the program checks the filter on every device at start-up.
- The searches use $S = \{2, 5\}$; the proofs hold for every S, and the tests were also run for five other sets ($\{2, 11\}$, $\{5, 11\}$, $\{2, 5, 11\}$, $\{2, 3, 5\}$, $\{3, 7\}$) without a failure [checked numerically], but on fewer polynomials than for $\{2, 5\}$.

Reproducing. The checks above are scripts of the repository, in `pdfilter_Test` unless stated; [1, Appendix B] gives the commands.

- tiny primes: `tiny_checks.sh`; the arithmetic and the size limits: `arith_checks.sh`;
- the oracle test and the faults: `fault_checks.sh` with `test_pdfilter_oracle.cpp`;
- the exact test: `poldisc_checks.sh`;
- the GPU filter: `test_gpu_filter.cpp`; the mathematics, the citations and the ledger: `checks.py`;
- the work units: `build/test-suite-unix.sh`.

Kept outside the repository: the captured candidates, the Linux build recipe used for the application with BOINC replaced, the Windows harness of the GPU runs, and the Barrett version, the instrumented copies and the scripts of the CPU timing.

References

[1] *Two Euclidean chains for one Legendre symbol: proofs and a worked course for the polDisc pre-filter of Passiflora*, `pdfilter_Test/pdfilter_chains.tex` in the Passiflora repository, 27 October 2026 (the source of every result here, with its proof).

[2] GetDecics, the NumberFields@home application, https://github.com/drivere/get-decics-numberfields (commit 638ca4e).

[3] H. Cohen, *A Course in Computational Algebraic Number Theory*, GTM 138, Springer, 1993.